

HelixCode

HelixCode

Распрацаваная платформа AI, якая падзяляе працу, захоўвае яе і ніколі не губляе ваша месца.

Зводка

HelixCode — гэта платформа распрацоўкі AI класа enterprise, заснаваная на Go, якая разбівае працэс распрацоўкі на інтэлектуальна размеркаваныя задачы паміж сеткай працоўных вузлоў, кіраваных SSH, з аўтаматычным стварэннем кантрольных кропак і адкатам, каб ніводная праца не была страчана. Яна аб'ядноўвае інтэграцыю з некалькімі LLM-правайдарамі, поўныя workflow жыццёвага цыклу распрацоўкі і крос-платформавую дастаўку праз інтэрфейсы REST, CLI, TUI і MCP.

Кароткае апісанне

HelixCode — гэта размеркаваная платформа распрацоўкі AI, напісаная на Go. Яна падзяляе працу на інтэлектуальныя задачы паміж сеткай працоўных вузлоў на базе SSH, захоўвае прагрэс з дапамогай аўтаматычнага стварэння кантрольных кропак і адкату, інтэгруе некалькі LLM-правайдараў і кіруе поўным жыццёвым цыклам распрацоўкі праз інтэрфейсы REST, CLI, TUI і MCP.

Падрабязнае апісанне

HelixCode — гэта размеркаваная платформа распрацоўкі AI класа enterprise (`dev.helix.code`, ліцэнзія MIT), створаная на аснове простага ідэі, якую яе дэвіз робіць літаральнай: падзяліць працу, захаваць яе і ніколі не губляць сваё месца. Яна прызначана для інтэлектуальнага падзелу задач, аўтаматычнага захавання працы і крос-платформавых workflow распрацоўкі, напісана на Go для забеспячэння

канкурэнтнасці і пераноснасці ў выглядзе адзінага бінарнага файла, што патрабуецца для размеркаваных вылічэнняў, — з аўтаматычным стварэннем кантрольных кропак, адкотам і маніторынгам у рэжыме рэальнага часу як асноўнымі прымітывамі, а не дадатковымі опцыямі.

Яе архітэктурна ўяўляе сабой паверхню API на аснове REST + WebSocket + MCP, якая абавязваецца на набор спецыялізаваных асноўных сэрвісаў: JWT-аўтэнтыфікацыю і кіраванне сесіямі, кіраванне пулам працоўных вузлоў на базе SSH з маніторынгам стану, кіраванне задачамі з кантрольнымі кропкамі і апрацоўкай залежнасцей, кіраванне праектамі і workflow, а таксама ўніфікаваны слой LLM-правайдараў. Усе гэта захоўваецца на PostgreSQL, прычым Redis даступны як апцыянальны ўзровень каардынацыі і кэшавання. Працоўныя вузлы аўтаматычна ўсталёўваюцца ў сетцы, таму маштабаванне кластара зводзіцца да таго, каб паказаць серверу на машыну, а не наладжваць яе ўручную. Шматкліенцкія інтэрфейсы ахопліваюць CLI, тэрмінальны інтэрфейс карыстальніка (TUI), REST і мабільныя фрэймворкі, што дазваляе атрымаць доступ да адной і той жа платформы з дапамогай скрыпта, тэрмінала ці дадатка.

HelixCode падтрымлівае поўны жыццёвы цыкл распрацоўкі ад пачатку да канца: планіраванне, стварэнне, тэсціраванне і рэфактарынг workflow выконваюцца аўтаматычна з улікам залежнасцей і кантэксту некалькіх сесіяў, таму доўгатэрміновая праца захоўвае сваю паслядоўнасць нават пры перапынках і змене машын. Яна інтэгруе некалькі LLM-правайдараў — llama.cpp, Ollama і OpenAI — праз адзіны інтэрфейс, дадае абвешчанае вызначэнне абсталявання, якое аналізуе даступныя рэсурсы (CPU/GPU/памяць) і падбірае мадэль пад канкрэтную машыну, а таксама падтрымлівае пашыраныя стратэгіі разважанняў, такія як “ланцужок думак” і “дрэва думак” для задач, якія патрабуюць больш за адзін праход. Model Context Protocol рэалізаваны праз некалькі транспартных пратаколаў для стандартызаванага абмену інструментамі і кантэкстам, а шматканальныя апавяшчэнні (Slack, Discord, Email, Telegram) трымаюць каманду ў курсе прагрэсу размеркаванай працы. Платформа арыентавана на Linux, macOS, Windows, Aurora OS і SymphonyOS.

Змест

Чаму мы яго стварылі

Распрацоўка з размеркаваннем і дапамогай AI звычайна губляе кантэкст і прагрэс, калі задачы разбіваюцца па машынах ці перарываюцца. HelixCode створаны, каб рабіць дзяленне задач разумным, а захаванне працы — аўтаматычным. Такім чынам вялікія распрацоўчыя намаганні

можна разбіць на часткі, размеркаваць па сетцы выканаўцаў, ствараць кантрольныя кропкі і аднаўляць ці адкочваць без страты стану.

Чаму гэта змяняе правілы гульні

Гэта робіць размеркаваную распрацоўку з AI *устойлівай* — магчымасць, якая раней была непрактычнай, калі каманды збіралі гэтыя элементы ўручную. Тры кампаненты, якія звычайна існуюць у трох асобных інструментах, аб'ядноўваюцца ў адной платформе: размеркаваныя вылічэнні (сеткі выканаўцаў SSH з аўтаматычнай ўстаноўкай і маніторынгам здароўя), дапамога ў распрацоўцы з AI (мультыправайдарскія LLM з разумовым аналізам і выклікам інструментаў) і аўтаматызацыя працоўных працэсаў на ўсім жыццёвым цыкле. Злучальным элементам з'яўляецца кантрольнае захаванне на аснове базы даных: паколькі стан задачы, кантрольныя кропкі і залежнасці захоўваюцца ў PostgreSQL, работа, якая ахоплівае многія машыны і сесіі, можа быць адкочана ці адноўлена менавіта з таго месца, дзе спынілася. Перапынкі і размеркаванне працы перастаюць быць крыніцай страчаных вынікаў і становяцца звычайнай, аднаўляльнай падзеяй.

Што новага

- Захаванне працы як асноўны прымітыў: аўтаматычнае кантрольнае захаванне і адкат, прымененыя да *размеркаваных* распрацоўчых задач, каб прагрэс заставаўся пры перапынках і збоях машын, а не знікаў разам з імі.
- Апаратна-арыентаваны выбар мадэлі, які аналізуе выяўленыя CPU/GPU/памяць і падбірае для кожнай задачы мадэль, якую машына можа эфектыўна выканаць — без ручной наладкі для кожнага выканаўца.
- Адна платформа, пяць уваходаў: REST, WebSocket, CLI, TUI і MCP, прычым MCP сама па сабе даступная праз некалькі транспартаў, каб інструменты і агенты маглі інтэгравацца незалежна ад спосабу падключэння.
- Крос-платфармавы ахоп, які выходзіць за межы звычайнай дэсктопнай тройкі і ўключае Aurora OS і SymphonyOS, пашыраючы сетку выканаўцаў на платформы, якія большасць інструментаў ігнаруе.

Галоўныя тэхнічныя выклікі і як мы іх пераадоолелі

- **Не губляць працу пры размеркаваных, перарывальных задачах.** Калі работа разбіваецца па машынах, любы збой ці перапынак звычайна пакідае ўсе незавершаныя часткі. Мы змадэлявалі саму задачу як носьбіт кантрольных кропак і залежнасцей, якія захоўваюцца ў PostgreSQL, каб сістэма магла вярнуцца да апошняга стабільнага стану ці аднавіць яго — устойлівасць, якая закладзена ў слой даных, а не ў крохкую апэратыўную памяць.
- **Кіраванне разнастайнай сеткай выканаўцаў.** Сетка машын на Linux, macOS, Windows, Aurora і SymphonyOS — гэта рухомы аб'ект з вечна зменлівай даступнасцю і настройкамі. Мы вырашаем гэта з дапамогай спецыяльнай службы пула выканаўцаў, якая ажыццяўляе рэгістрацыю на аснове SSH, аўтаматычную ўстаноўку на новыя вузлы і бесперапынны маніторынг здароўя, каб сетка заставалася кантралюемай і вядомай, нягледзячы на змены ў складзе машын.
- **Разнастайнасць правайдараў і абсталявання.** LLM-бэкэнды і машыны, якія іх выконваюць, моцна адрозніваюцца па магчымасцях. Мы схавалі гэта за адзіным інтэрфейсам правайдара LLM і спалучылі яго з выяўленнем абсталявання (CPU/GPU/памяць), якое забяспечвае разумны выбар мадэлі, каб правільная мадэль трапляла на правільную машыну без неабходнасці разважаць над гэтым карыстальніку.

Змесціва

Тэхналагічны стэк

- **Go (1.26+ унутраны модуль)** — абраны таму, што яго канкурэнтнасць на аснове гаруцін і вывад у выглядзе адзінага бінарнага файла — гэта менавіта тое, што патрэбна размеркаванай сістэме працоўных вузлоў: танная паралельнасць для аркестрацыі і аўтаномны бінарны файл, які аўтаматычна ўсталёўваецца на любы вузел. Ён змяшчае ўсе асноўныя службы і бінарныя файлы CLI/server.
- **Gin (фрэймворк HTTP)** — абраны дзеля хуткага і мінімалістычнага REST-слоя з нізкімі накладнымі выдаткамі; ён абслугоўвае паверхню /api/v1 (аўтэнтыфікацыя, працоўныя вузлы, задачы, праекты), з якой узаемадзейнічае кожны кліент.
- **PostgreSQL 15+ (праз pgx/v5)** — абраны як надзейная сістэма захоўвання даных, паколькі кантрольныя кропкі і адкат патрабуюць транзакцыйнай устойлівасці; ён змяшчае

схему размеркаваных вылічэнняў з 11 табліц (карыстальнікі, працоўныя вузлы, задачы, праекты, сесіі, пастаўшчыкі LLM, апавяшчэнні), якая забяспечвае захаванне працы.

- **Redis 7+ (факультатыўна, go-redis/v9)** — абраны як факультатыўны ўзровень кэшавання і каардынацыі, які паскарае гарачыя шляхі без ператварэння ў жорсткую залежнасць, так што мінімальнае разгортванне можа працаваць толькі на Postgres.
- **SSH** — абраны як транспарт кіравання працоўнымі вузламі менавіта таму, што ён ужо паўсюдна распаўсюджаны і ўжо абаронены; ён кіруе рэгістрацыяй вузлоў, аўтаматычнай усталёўкай і дыстанцыйным выкананнем каманд па ўсёй сетцы без неабходнасці папярэдняга разгортвання спецыяльнага агента.
- **Model Context Protocol (MCP)** — абраны для стандартызаванага абмену інструментамі і кантэкстам, каб знешнія інструменты і агенты інтэгралізіраваліся праз адзін адкрыты пратакол; рэалізаваны з падтрымкай некалькіх транспартаў, каб адпавядаць кліентам незалежна ад спосабу падлучэння.
- **Пастаўшчыкі LLM (Llama.cpp, Ollama, OpenAI)** — абраны для ахоплення як лакальнага, так і хостынгавага інферэнсу праз адзіны інтэрфейс, каб залежнае ад абсталявання вызначэнне маршруту магло накіроўваць задачу на лакальную мадэль ці хостынгавую без ведама кліента.

Стан і нататкі аб шчырасці

- **Стан: бэта-версія.** У README паведамляецца пра стан “ПОЎНАСЦЮ ЗАВЕРШАНЫ / усе 5 этапаў”; гэтая завершанасць заяўлена праектам, а не незалежна пацверджана, таму старонка разглядае яго як бэта-версію.
- Усе вышэйзгаданыя дэталі паходзяць з README рэпазіторыя; маркетынгавыя фармулёўкі (слоганы) з’яўляюцца рэдакцыйнымі і не з’яўляюцца паказчыкамі з крыніцы.

Прыярытэтны ўзровень: Helix-першасны.